

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет Компьютерных наук

Кафедра программирования и информационных технологий

Мобильное приложение ZadachOk для распределения семейных обязанностей

Курсовая работа

Направление: 09.03.04. Программная инженерия

Зав. Кафедрой _____ д. ф.-м. н, доцент С.Д. Махортов
Руководитель _____ ст. преподаватель В.С. Тарасов
Руководитель практики _____ Е.Д. Проскуряков
Обучающийся _____ В.С. Мосякин, 3 курс, д/о
Обучающийся _____ И.В. Четвергов, 3 курс, д/о
Обучающийся _____ К.А. Лышова, 3 курс, д/о
Обучающийся _____ А.В. Зыков, 3 курс, д/о
Обучающийся _____ А.В. Беляев, 3 курс, д/о
Обучающийся _____ Д.В. Круглов, 3 курс, д/о

Воронеж 2025

СОДЕРЖАНИЕ

Определения, обозначения и сокращения	3
Введение	4
1 Постановка задачи	5
1.1 Цели создания системы.....	5
1.2 Функциональные требования к разрабатываемой системе	7
1.3 Нефункциональные требования к разрабатываемой системе	7
2 Анализ предметной области	10
3 Реализация	11
3.1 Frontend.....	11
3.1.1 Локальное состояние (виджетный уровень)	11
3.1.2 Глобальное состояние (прикладной уровень)	11
3.1.3 Персистентное состояние (хранение данных)	12
3.1.4 Взаимодействие с серверной частью.....	13
3.1.5 Структура проекта.....	13
3.2 Backend	16
3.2.1 Общая архитектура	16
3.2.2 Контроллеры	16
3.2.3 Сервисный уровень	17
3.2.4 Репозитории и доступ к данным	17
3.2.5 Обработка ошибок и безопасность	17
3.3 БД и СУБД	18
3.3.1 Описание базы данных AC ZadachOk	18
3.3.2 Безопасность, производительность и резервирование.....	19
3.4 Интеграция искусственного интеллекта (переименовать)	21
3.5 Развёртывание и запуск (переименовать)	21
3.5.1 Процесс развертывания	21
3.5.2 Финальная архитектура и реализация	21
Заключение.....	23
Список использованных источников	24

Определения, обозначения и сокращения

Термин	Определение

Введение

В современном мире самоорганизация и распределение обязанностей в семье становятся всё более актуальными. Многие люди сталкиваются с проблемой забывчивости, отсутствия мотивации и неэффективного планирования совместных задач. Существующие **таск-трекеры** часто не учитывают семейный контекст и не предоставляют инструментов для мотивации участников.

Система предназначена для совместного ведения списка задач с возможностью создания групп пользователей. **В зависимости от установленных администратором условий**, участники могут получать вознаграждение в виде внутренней некоммерческой валюты. В приложении предусмотрен встроенный магазин, в котором данная валюта может быть использована для приобретения доступных товаров или услуг.

Приложение ЗадачОк решает эти проблемы, предлагая:

- Распределение задач между членами семьи;
- Геймификацию с системой вознаграждений;
- Умные уведомления на основе ИИ для повышения вовлечённости;

Проект актуален, так как люди часто игнорируют стандартные напоминания. ИИ генерирует персонализированные и шутливые сообщения, повышая внимание пользователей. Также к этому можно отнести систему баллов и виртуального магазина, которая стимулирует пользователя нашего приложения к выполнению задач. К тому же стоит отметить, что большинство таск-трекеров не адаптированы для семейного использования, что ставит наше приложение в более выигрышную позицию на рынке нашей **ЦА**.

1 Постановка задачи

1.1 Цели создания системы

- В течение первых трех месяцев после запуска приложения планируется обеспечить привлечение 100 активных пользователей, под которыми понимаются регулярно взаимодействующие с системой лица, выполняющие ключевые действия: создание и завершение задач, использование виртуального магазина и участие в системе вознаграждений. Достижение указанного показателя будет осуществляться через комплексное продвижение в цифровых каналах, включая социальные сети и тематические площадки, с параллельной оптимизацией позиционирования в магазинах приложений. Особое внимание уделяется органическому росту аудитории за счет реферальных механизмов и пилотного тестирования среди потенциально заинтересованных пользователей. Мониторинг эффективности кампании будет проводиться на основе анализа динамики установок, показателей вовлеченности и коэффициента удержания, с периодической корректировкой стратегии продвижения. Прогнозируемые значения базируются на исследовании аналогичных решений в сегменте семейных организационных приложений;
- В рамках исследования будет проведено сравнительное тестирование разработанного таск-трекера ЗадачОк с аналогами конкурентов на тестовой группе из 4 пользователей. Методика предполагает попеременное использование продуктов в равных временных промежутках с фиксацией показателей своевременного выполнения задач. Критерием эффективности решения является достижение уровня не менее 50% выполненных в срок задач при использовании ЗадачОк, что должно подтвердить преимущества применяемых подходов, включая игровые механизмы мотивации и

персонализированные уведомления. Для обеспечения достоверности результатов используется метод попеременного тестирования с комплексной оценкой количественных и качественных показателей, позволяющий сделать обоснованные выводы о результативности предложенного решения для организации семейных задач;

- В течение шести месяцев после публикации приложения в **Play Market** планируется достичь конверсии 5% пользователей в платящих клиентов за счет внедрения **премиум-подписки (см. ПРИЛОЖЕНИЕ ...)** (99 руб./месяц или 799 руб./год) с расширенным функционалом и рекламной модели с партнерской интеграцией. Ключевые показатели эффективности включают удержание платящих пользователей на уровне 60%, средний доход 35 руб./пользователя в месяц и соотношение **LTV/CAC** ≥ 3 . Для достижения цели предусмотрено А/В-тестирование механик монетизации, поэтапное введение платных функций и партнерская программа, при этом базовый функционал остается бесплатным. Прогноз основан на анализе аналогичных приложений с аудиторией от 10 000 установок.

1.2 Функциональные требования к разрабатываемой системе

- Создание и управление задачами;
- Система баллов и управление виртуальным магазином;
- Генерация текста для уведомлений с использованием (на основе) ИИ;
- Поддержка офлайн-режима;
- Ещё что-то про профиль пользователя мб + про статистику по задачам.

1.3 Нефункциональные требования к разрабатываемой системе

Производительность:

- Время загрузки календаря и списка задач не должно превышать 30 секунд при стабильном интернет-соединении со скоростью не менее 5 Мбит/с;
- Синхронизация данных между устройствами должна завершаться в течение 60 секунд после восстановления сетевого соединения;
- Операции в оффлайн-режиме (просмотр задач, отметка о выполнении) должны выполняться мгновенно, в зависимости от устройства, без ощутимых задержек (см. мин. сис. требования);
- Формирование статистики по выполненным задачам (графики, отчеты) должно занимать не более 15 секунд;
- Отображение содержимого виртуального магазина должно происходить в течение 20 секунд после открытия раздела.

Надежность:

- Система должна обеспечивать бесперебойную работу 70% времени в течение суток;
- Автоматическое сохранение данных должно происходить каждые 5 минут при активной сессии;

- В случае сбоя должно сохраняться не менее 70% пользовательских данных;
- Логирование всех критических ошибок с фиксацией времени, типа ошибки и контекста возникновения;
- Возможность восстановления данных из резервной копии за период не более 24 часов.

Безопасность:

- Все передаваемые данные должны шифроваться по протоколу HTTPS;
- Пароли пользователей должны храниться в хешированном виде;
- Сессионные токены должны иметь срок действия не более 24 часов;
- Защита от SQL-инъекций и XSS-атак на уровне серверного API.

Совместимость:

- Поддержка Android версии 13 и выше;
- Адаптация интерфейса для экранов с разрешением от 720x1280 до 1440x2560 пикселей;
- Корректная работа на устройствах с объемом оперативной памяти от 2 ГБ;

Тестируемость:

- Покрытие unit-тестами не менее 50% критического функционала
- Поддержка не менее 3 различных тестовых конфигураций устройств;
- Возможность тестирования в эмуляторах с разными характеристиками;
- Наличие инструментов для нагрузочного тестирования API;

- Масштабируемость:
- Возможность одновременной работы не менее 1000 активных пользователей;
- Поддержка увеличения количества групп до 100 без потери производительности;
- Гибкая система кэширования часто запрашиваемых данных;
- Возможность горизонтального масштабирования серверной части.

2 Анализ предметной области

3 Реализация

3.1 Frontend

Фронтенд-часть приложения реализована на кроссплатформенном фреймворке Flutter с использованием языка программирования Dart. Данный выбор обусловлен следующими преимуществами:

- Поддержка горячей перезагрузки (Hot Reload) для ускорения процесса разработки;
- Богатая экосистема готовых компонентов и библиотек;
- Высокая производительность за счет компиляции в нативный код.

3.1.1 Локальное состояние (виджетный уровень)

Для управления локальным состоянием интерфейсных компонентов применяется механизм `StatefulWidget` в сочетании с методом `setState()`. Данный подход обеспечивает реактивное обновление пользовательского интерфейса при изменении внутреннего состояния виджетов. Особое внимание уделяется корректной обработке жизненного цикла компонентов - перед каждым обновлением выполняется проверка флага `mounted`, что предотвращает попытки обновления уже удаленных из дерева виджетов. Этот уровень управления состоянием преимущественно используется для обработки пользовательских взаимодействий с формами ввода, управления видимостью вспомогательных элементов интерфейса и обработки временных состояний, связанных с конкретными экранами приложения. Реализация включает контроль за созданием и редактированием задач, переключением настроек интерфейса и динамическим отображением пользовательского контента, **такого как аватар профиля.**

(дополнить/убрать)

3.1.2 Глобальное состояние (прикладной уровень)

Глобальное состояние приложения организовано с использованием библиотеки `Provider`, которая реализует паттерн `InheritedWidget`. Данное

решение позволяет централизованно управлять критически важными данными приложения, такими как информация о текущем пользователе, настройки системы и состояние групп задач. Архитектура построена на выделенных классах-провайдерах (AuthProvider, SettingsProvider, GroupProvider, TaskProvider), каждый из которых отвечает за определенную предметную область. Провайдеры обеспечивают механизм подписки на изменения состояния и автоматическое распространение обновлений по всему приложению. Особенностью реализации является строгое разделение бизнес-логики и представления, что повышает тестируемость кода и упрощает его поддержку. Данный уровень управления состоянием используется для хранения и обработки данных, которые должны быть доступны из различных частей приложения. **Описать здесь ещё про обращения на сервер через api, мол что именно здесь реквест на сервак запрашивается**

(дополнить/убрать)

3.1.3 Персистентное состояние (хранение данных)

Для сохранения состояния между сессиями приложения используется механизм персистентного хранения данных на основе библиотеки `shared_preferences`. Реализация обеспечивает надежное сохранение критически важных настроек пользователя, включая параметры авторизации (токены доступа), предпочтения интерфейса (настройки уведомлений) и локальный кэш часто используемых данных. Особенностью данного подхода является автоматическое восстановление состояния при повторном запуске приложения, что создает эффект непрерывности пользовательского опыта. Все сохраняемые данные проходят процедуру **сериализации/десериализации**, обеспечивающую их целостность. Для защиты конфиденциальной информации применяются механизмы шифрования, предоставляемые платформой. Реализация учитывает ограничения мобильных устройств по объему хранимых данных и оптимизирована для работы с небольшими объемами структурированной информации.

(дополнить/убрать)

3.1.4 Взаимодействие с серверной частью

Взаимодействие между клиентской и серверной частями приложения организовано через набор четко определенных API-эндпоинтов, соответствующих REST-архитектуре. Для каждого экрана реализован специализированный набор запросов: экран регистрации (RegisterScreen) использует POST-запрос для создания нового пользователя, экран задач (TasksScreen) взаимодействует с API через GET-запросы для получения списка задач и POST-запросы для их создания/обновления. Все передаваемые данные сериализуются в формат JSON и защищаются с помощью протокола HTTPS. Авторизация пользователей осуществляется через механизм JWT, где токен доступа автоматически добавляется в заголовки каждого запроса после успешной аутентификации. Для обработки асинхронных операций используется механизм Future в сочетании с обработчиками состояний (loading, error, success), что обеспечивает отзывчивый интерфейс даже при выполнении сетевых операций. Особенностью реализации является двухуровневая система кэширования - локальное хранение актуальных данных в провайдерах и их периодическая синхронизация с сервером, что позволяет сохранять функциональность приложения в офлайн-режиме. Все ошибки сетевого взаимодействия логируются и обрабатываются с выводом соответствующих уведомлений пользователю.

(дополнить/убрать)

3.1.5 Структура проекта

Архитектура клиентской части приложения организована по модульному принципу с четким разделением на слои, что соответствует современным подходам к разработке мобильных приложений. Основные компоненты структуры проекта включают следующие элементы:

3.1.5.1 Слои приложения

Проект разделен на три основных слоя: presentation (представление), business logic (бизнес-логика) и data (работа с данными). В слое представления находятся все виджеты и экраны приложения, такие как TaskScreen и RegisterScreen, которые отвечают исключительно за отображение информации и обработку пользовательских взаимодействий. Слой бизнес-логики содержит провайдеры (Provider), включая TaskProvider, GroupProvider и AuthProvider, которые инкапсулируют всю бизнес-логику приложения и управляют его состоянием. Слой работы с данными включает модели (например, TaskModel, UserModel) и репозитории, отвечающие за взаимодействие с API и локальным хранилищем.

(дополнить/убрать)

3.1.5.2 Организация экранов

Каждый экран приложения представляет собой самостоятельный модуль, содержащий весь необходимый код для его функционирования. Например, экран задач (TaskScreen) включает не только UI-компоненты, но и обработчики событий, валидацию данных и логику взаимодействия с провайдерами. Для повторно используемых компонентов (кнопки, поля ввода, карточки задач) созданы отдельные виджеты, что обеспечивает единообразие интерфейса и упрощает поддержку кода.

(дополнить/убрать)

3.1.5.3 Управления зависимостями

Все внешние зависимости и настройки темы вынесены в отдельные файлы конфигурации. Константы, используемые в интерфейсе (цвета, отступы, размеры шрифтов), определены в специализированных классах (например, TaskScreenConstants), что позволяет централизованно управлять визуальным стилем приложения. Для работы с API создан отдельный модуль (ApiInterface), который инкапсулирует всю логику сетевых запросов.

(дополнить/убрать)

3.1.5.4 Маршрутизация

Навигация между экранами организована через именованные маршруты с использованием Navigator 2.0, что обеспечивает предсказуемое поведение приложения и возможность глубоких ссылок. Все маршруты зарегистрированы в централизованном файле маршрутизации, что упрощает их модификацию и добавление новых экранов.

(дополнить/убрать)

3.1.5.5 Обработка состояний

Для управления состоянием приложения используется комбинация локального (StatefulWidget) и глобального (Provider) подходов. Критически важные данные (информация о пользователе, список задач, настройки групп) хранятся в провайдерах и автоматически синхронизируются между различными экранами приложения. Временные состояния (например, заполнение форм) управляются на уровне отдельных виджетов.

(дополнить/убрать)

3.1.5.6 Тестирование и отладка

Архитектура проекта предусматривает возможность модульного тестирования всех компонентов. Ключевые бизнес-процессы (регистрация пользователя, создание задач, управление группами) вынесены в отдельные методы, что позволяет легко тестировать их изолированно. Для отладки сетевых запросов реализовано логирование всех входящих и исходящих данных.

(дополнить/убрать)

3.2 Backend

3.2.1 Общая архитектура

Серверная часть приложения разработана по архитектурному шаблону «трёхуровневая архитектура», где каждый уровень несёт строго определённую функциональную нагрузку. Основное разделение логики реализовано между уровнями контроллеров, сервисов и репозиториев. Данный подход способствует модульности, повторному использованию кода, а также облегчает сопровождение и расширение программной системы.

Контроллеры отвечают за приём HTTP-запросов от клиента, их валидацию и передачу обработанных данных на уровень бизнес-логики. Сервисный слой инкапсулирует основную логику обработки информации и взаимодействует с уровнем репозиториев, который, в свою очередь, обеспечивает доступ к базе данных посредством использования механизмов JPA (Java Persistence API) и аннотаций Spring Data.

(дополнить/убрать)

3.2.2 Контроллеры

Контроллеры реализуют API-интерфейс системы и выступают в роли посредника между клиентской частью и бизнес-логикой. Каждый контроллер соответствует определённой сущности предметной области, такой как пользователь (Customer), задача (Task), магазин (Shop), кошелёк (Wallet), лобби (Lobby) и т.д. В их обязанности входит получение параметров запроса, проверка корректности полученных данных, а также формирование и возврат ответа в формате JSON.

Обработка запросов осуществляется с использованием аннотаций Spring Framework, таких как `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping` и `@DeleteMapping`. Каждый метод контроллера соответствует отдельной операции CRUD (создание, чтение, обновление, удаление), реализуемой над сущностью

(дополнить/убрать)

3.2.3 Сервисный уровень

Сервисный уровень является центральным компонентом бизнес-логики приложения. В данном слое сосредоточена обработка входных данных, реализуются проверки условий, обеспечивается согласованность данных и осуществляется вызов методов, реализующих взаимодействие с хранилищем данных. Сервисный уровень получает данные от контроллеров и, при необходимости, передаёт их на уровень репозитория.

Реализация сервисов осуществляется посредством интерфейсов и их реализаций, помеченных аннотациями `@Service`. Подобная структура способствует расширяемости и удобству внедрения зависимости (Dependency Injection), что реализуется средствами Spring Container.

(дополнить/убрать)

3.2.4 Репозитории и доступ к данным

Уровень репозитория предоставляет абстракцию над базой данных и реализует механизмы хранения, поиска и удаления информации. Использование Spring Data JPA позволяет упростить взаимодействие с базой данных за счёт автоматической генерации SQL-запросов на основе имён методов. Каждой сущности в проекте соответствует отдельный интерфейс-репозиторий, расширяющий интерфейс `JpaRepository`, что даёт доступ ко множеству базовых операций.

Дополнительно в некоторых случаях определяются собственные запросы, реализуемые с помощью аннотаций `@Query`, или используются механизмы Criteria API для более сложных условий фильтрации и агрегации данных.

(дополнить/убрать)

3.2.5 Обработка ошибок и безопасность

Особое внимание при проектировании backend-части уделено обработке ошибок. Для перехвата исключений, возникающих на различных уровнях приложения, реализован глобальный обработчик ошибок, использующий

аннотацию `@ControllerAdvice`. Он позволяет централизованно управлять кодами HTTP-ответов и сообщениями об ошибках.

Кроме того, реализована система авторизации и аутентификации с использованием JWT (JSON Web Token). Каждый пользователь после успешного входа получает токен доступа, который должен быть прикреплен к каждому последующему запросу для подтверждения подлинности.

(дополнить/убрать)

3.3 БД и СУБД

3.3.1 Описание базы данных AC ZadachOk

База данных автоматизированной системы ZadachOk спроектирована в соответствии с требованиями, изложенными в техническом задании, и реализована на основе реляционной модели данных. В качестве основной системы управления базами данных выбрана PostgreSQL версии 17, что обусловлено требованиями к надежности, производительности и масштабируемости системы.

Архитектура базы данных включает комплекс взаимосвязанных сущностей, отражающих предметную область системы распределения семейных обязанностей. Центральной сущностью является Пользователь (Customer), который содержит уникальный идентификатор, персональные данные (имя, электронная почта), аутентификационные данные (хеш пароля), дату рождения и флаг администратора. Каждый пользователь может состоять в нескольких группах (Lobby), что реализовано через отношение многие-ко-многим.

Сущность Группа (Lobby) представляет собой объединение пользователей для совместного выполнения задач и включает идентификатор, название, описание и дату создания. В рамках группы создаются и выполняются задачи (Task), которые содержат информацию о названии, описании, статусе выполнения, сроке исполнения и размере вознаграждения.

Каждая задача принадлежит конкретной группе и может быть назначена на определенного пользователя.

Финансовый аспект системы реализован через сущность Кошелек (Wallet), который хранит информацию о балансе пользователя в рамках конкретной группы. Изменение баланса происходит при выполнении задач (пополнение) и совершении покупок в виртуальном магазине (списание). Сущность Магазин (Shop) привязана к группе и содержит перечень доступных товаров (Product) с указанием их стоимости в баллах, описания и изображения. (дополнить/убрать).

3.3.2 Безопасность, производительность и резервирование

Система ограничений базы данных гарантирует сохранение целостности при всех операциях. Реализованы каскадные обновления и удаления для связанных данных, проверки значений полей и уникальности ключевых атрибутов. Особое внимание уделено безопасности: пароли хранятся в хешированном виде, персональные данные шифруются, а все соединения используют защищенные протоколы. Разграничение доступа реализовано через комбинацию ролевой модели и ограничений на уровне SQL-запросов.

Для обеспечения высокой производительности в базе данных созданы составные индексы на часто используемых полях, оптимизированы наиболее ресурсоемкие запросы. Архитектура предусматривает возможность шардинга данных пользователей при росте нагрузки. Реализовано кэширование результатов сложных запросов, таких как статистика выполнения задач. Эти решения позволяют системе сохранять отзывчивость при увеличении количества пользователей до 10 000 активных сессий.

Система миграций на основе Flyway обеспечивает контроль версий схемы базы данных и возможность отката изменений. Ежедневные полные бэкапы и почасовые инкрементальные копии гарантируют восстановление данных при любых сбоях. Процедуры резервного копирования включают проверку целостности сохраненных данных и автоматическое оповещение

администратора в случае обнаружения проблем. Все изменения в структуре базы данных фиксируются в журнале изменений с указанием автора и цели модификации.

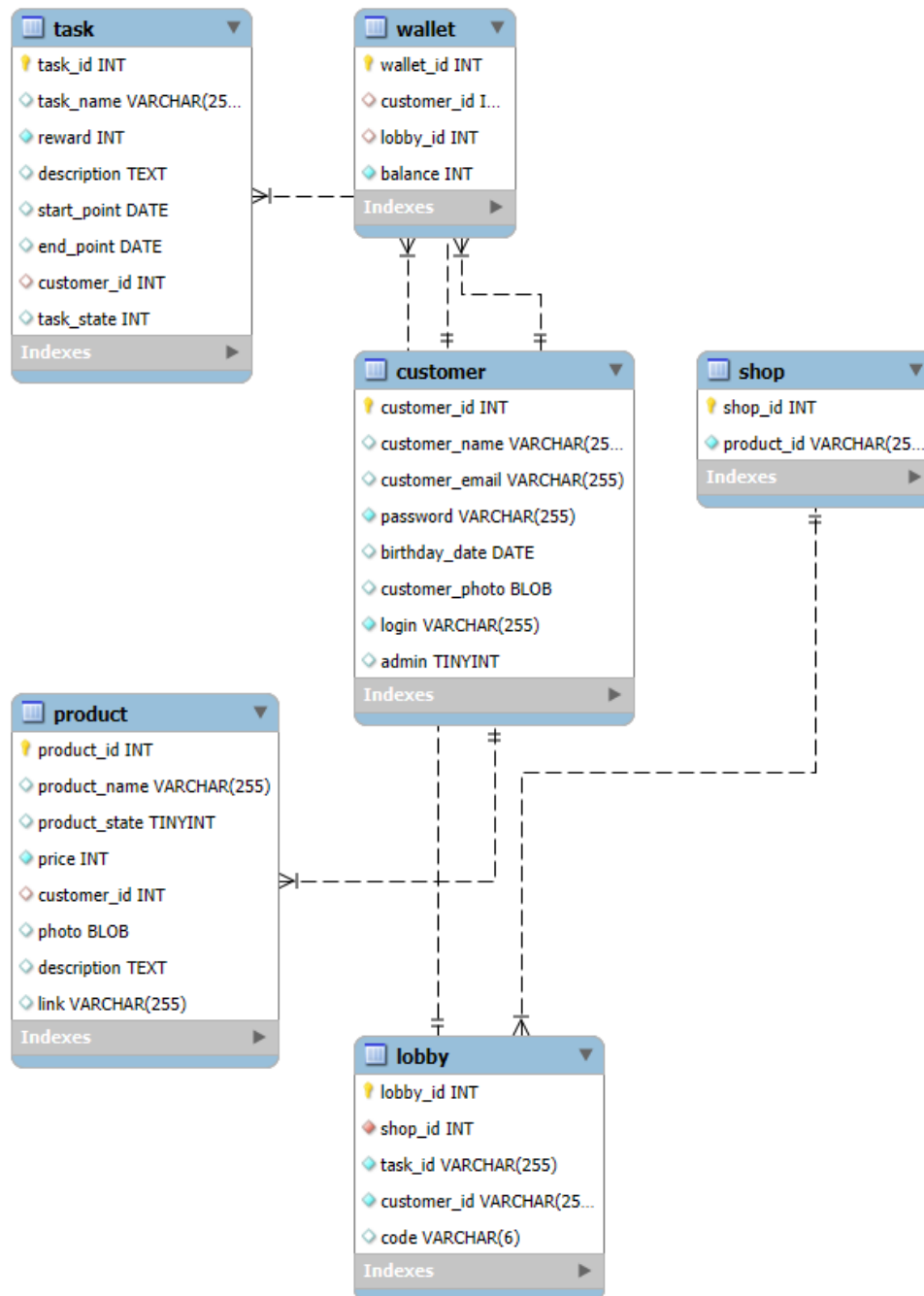


Рисунок 1 — ER диаграмма.

(дополнить/убрать)

3.4 Интеграция искусственного интеллекта (переименовать)

3.5 Развёртывание и запуск (переименовать)

3.5.1 Процесс развертывания

Наш путь к успешному развертыванию системы прошел через несколько ключевых этапов, каждый из которых привнес ценные уроки в финальную конфигурацию. Первоначальная попытка использовать Docker столкнулась с фундаментальными проблемами сетевой конфигурации - порты API оказались недоступны извне из-за ограничений хостинг-провайдера, а внутренняя маршрутизация между контейнерами работала некорректно из-за неправильно настроенной сети Docker. Эти проблемы вынудили нас временно отказаться от контейнеризации в пользу ручного развертывания.

Ручная настройка серверного окружения выявила новые вызовы. Потребовалась тщательная настройка systemd для управления Java-процессом API, включая оптимизацию параметров JVM под доступные ресурсы сервера. Конфигурация Nginx как reverse проху потребовала глубокого понимания механизмов SSL/TLS и особенностей проксирования запросов. Однако самым серьезным испытанием стало обнаружение кибератаки, когда сервер был скомпрометирован через уязвимость в системе безопасности. Этот инцидент подчеркнул критическую важность своевременного обновления, мониторинга и жесткой настройки прав доступа. Тут надо всё более официально написать без нас и прочего

(дополнить/убрать)

3.5.2 Финальная архитектура и реализация

Извлеченные уроки легли в основу окончательной архитектуры решения. Мы вернулись к Docker, но с совершенно новым подходом к безопасности и надежности. Контейнер PostgreSQL был настроен с персистентным хранилищем данных и строгими ограничениями доступа. Java-API развертывался с минимально необходимыми привилегиями, а все чувствительные данные (такие как пароли БД) вынесены в Docker Secrets.

Nginx получил усиленную конфигурацию с TLS 1.3, HTTP/2 и строгими политиками безопасности.

Особое внимание уделили сетевой изоляции - все контейнеры работают в `dedicated` сети Docker с явно заданными правилами взаимодействия. Для мониторинга состояния системы внедрили комбинацию логирования и метрик, что позволяет оперативно обнаруживать аномалии. Процесс CI/CD был полностью переработан - теперь каждый коммит проходит через `pipeline` с тестами, проверкой безопасности образов и автоматическим развертыванием на `staging`-окружении перед продам.

Результатом стала отказоустойчивая система, сочетающая преимущества контейнеризации с надежностью, проверенной в ходе предыдущих испытаний. Финальная архитектура не только решает первоначальные задачи, но и предусматривает возможности для горизонтального масштабирования и бесперебойного обновления - ключевые требования для будущего роста проекта.

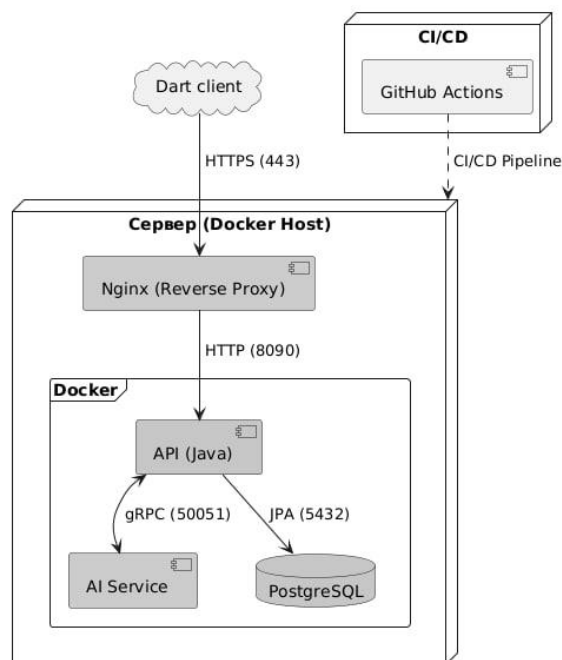


Рисунок 2 — Схема развертывания.

(дополнить/убрать)

Заключение

Разработано мобильное приложение для распределения семейных задач с уникальными функциями:

Геймификация.

ИИ-уведомления.

Перспективы: интеграция с реальными магазинами, расширение ролей.

(дополнить/убрать и отформатировать)

Список использованных источников

- Официальная документация Flutter SDK;
- Официальная документация Dart;
- Официальная документация Java;
- Официальная документация Lombok;
- Официальная документация Spring Boot;
- Официальная документация PostgreSQL;
- ГОСТ 34.201-20. Информационная технология. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированной системы;
- ГОСТ 2.105-19. ЕСКД. Общие требования к текстовым документам.

(дополнить/убрать и отформатировать)

ПРИЛОЖЕНИЕ А

Сюда по-хорошему пихнуть диаграммку, брендбук и айконкит.